

DELIVERY AND REPOSITORY PLAN

AssetPulse Lite

Eight-week incremental delivery and Git plan

Build the approved AssetPulse Lite contract in genuine vertical increments. Keep main understandable, tested, and deployable. Tag a checkpoint only when its real exit gate passes.

Decision	Execution plan
Status	v0.5 complete; only v0.6, v0.7, and v1.0 remain
Duration	Eight focused delivery phases
Checkpoints	v0.1 through v0.7, then v1.0
Working model	Issues, short branches, focused pull requests, required checks, and frequent pushes
Hosting boundary	\$0 / no-card public portfolio deployment; current plan is Render Free plus Neon Free
Controlling scope	AssetPulse Lite Product and Engineering Delivery Contract v1.1

A checkpoint is complete only when behavior, tests, and delivery evidence can be inspected.

01 | WORKING RULES

Working rules

Incremental delivery without manufactured history.

- Use one issue for each coherent outcome or defect, with contract IDs and acceptance notes.
- Use short-lived branches and keep main buildable after v0.1.
- Commit coherent outcomes after relevant checks pass and push after meaningful sessions.
- Keep implementation, tests, and directly affected documentation together.
- Use follow-up commits only for real integration fixes, review changes, or refactoring.
- Do not backdate, fabricate defects, create noise commits, or rewrite history to cosmetically remove earlier decisions.
- Tag only after the checkpoint gate passes; calendar completion alone is not enough.
- Never create a paid subscription or enter payment details for the portfolio deployment.

Standard change cycle

Step	Required action
1. Issue	State outcome, contract IDs, checks, and security / tenant effect.
2. Branch	Create a short descriptive branch.
3. Increment	Implement the smallest coherent behavior that builds or runs.
4. Evidence	Add focused tests and at least one relevant failure or denial.
5. Integrate	Run locally and update the public environment only when the phase calls for it.
6. Pull request	Describe outcome, evidence, limitations, migration, and deployment impact.
7. Merge / release	Merge green work; deploy and tag only at the real exit gate.

Release map

Phase	Primary outcome	Status
v0.1	Runnable skeleton, CI, authentication, tenants, and protected asset view.	Complete
v0.2	Seeded asset / sensor / rule domain and tenant-safe reads.	Complete
v0.3	Atomic idempotent telemetry, simulator, and visible readings.	Complete
v0.4	Durable processing, retry / dead state, alerts, and SSE.	Complete
v0.5	Work orders, ownership, audit / history, and optimistic conflict.	Complete
v0.6	Integrated UX, security matrix, accessibility, and three journeys.	Remaining
v0.7	\$0 public hosting, observability, scans, performance, smoke, and rollback.	Remaining
v1.0	Release correction, evidence package, demo, screenshots, and immutable release.	Remaining

02 | COMPLETED PHASES

Completed phases

The plan records what is already implemented without reopening finished work.

Checkpoint	Implemented outcome	Accepted evidence
v0.1	Reproducible application, migrations, seeded session auth, tenant context, protected asset list, Compose, and CI.	Focused suites and exact-source local runtime.
v0.2	Relational seeded assets / sensors / rules and tenant-safe read-only detail.	Database constraints, migration tests, positive and negative API / UI evidence.
v0.3	Atomic idempotent telemetry, bounded range query, simulator scenarios, chart / table surface.	Retry, rollback, isolation, simulator, and packaged flow evidence.
v0.4	Durable worker leasing and retry, dead-state operations, deterministic alerts, state history, and authorised SSE.	Restart / concurrency tests, exact-source smoke, and two-client browser evidence.
v0.5	Alert-to-work-order creation, assignment, technician lifecycle, immutable history, audit, and stale-client recovery.	Merged GitHub-hosted CI plus exact-source local packaged, browser, database, denial, and conflict evidence.

v0.5 acceptance decision

The dispatcher-to-technician path, illegal / foreign denial with no history, and two-stale-client one-success / one-conflict recovery are implemented and evidenced. A public demo is not claimed at this checkpoint. The earlier requirement to host each weekly slice is replaced by one deliberate zero-cost public deployment in v0.7, after product assurance is complete.

Evidence integrity

- Green CI identifies the integrated source revision and preserves the four required job names.
- Local packaged evidence uses images rebuilt from the exact source under review.
- Documentation distinguishes implemented behavior from future public-host behavior.
- Earlier Git history is preserved. Current files, goals, and delivery automation control future work.

03 | V0.6 - PRODUCT ASSURANCE

v0.6 - Product assurance

Remaining phase one of three: integrate the experience and prove the core misuse cases.

Outcome

The core workflow is coherent, backend-enforced, accessible in its primary paths, and demonstrated by three end-to-end journeys.

Increment sequence

- Complete dashboard counts, recent activity, and required loading / empty / error / forbidden states.
- Complete session / CSRF, login rate limiting, security headers, validation, and log-redaction checks.
- Finish the tenant / role / ownership matrix and direct backend denial evidence.
- Implement live-incident, isolation / role, and concurrency Playwright journeys.
- Run keyboard and accessibility checks on the core screens and retain a chart table fallback.
- Add a bounded demo reset. Keep its public execution evidence pending until v0.7 is live.

Exit gate

Three local journeys pass, direct forbidden requests fail safely, primary flows remain keyboard usable, and no critical accessibility issue remains. The live-incident script is ready to run unchanged against the v0.7 public URL.

No scope expansion

v0.6 does not add hosting accounts, observability vendors, advanced dashboard analytics, or optional product capability.

04 | V0.7 - ZERO-COST PUBLIC OPERATIONS

v0.7 - Zero-cost public operations

Remaining phase two of three: publish, measure, secure, and document the portfolio release without a paid subscription.

Target topology

- Render Free runs one Docker web service containing Nginx and Spring Boot. Nginx is public; Spring listens only on loopback.
- Neon Free provides PostgreSQL over a verified TLS connection. A SQL-created least-privilege role is used by the app.
- GitHub required checks gate main. Render deploys main only after checks pass.
- The repository contains no secrets. The owner supplies database values directly in the hosting dashboard.
- No payment method, paid trial, paid subscription, or automatic overage is accepted.

Increment sequence

- Validate the composite non-root image under the same-origin public smoke contract.
- Create the Neon Free project and least-privilege application role; record the bounded seed / reset procedure.
- Create the Render Free Blueprint service and verify commit / image identification.
- Run status, authentication, tenant denial, logout, simulator, live incident, and SSE reconnect smoke against the public URL.
- Add structured logs and four operational metrics; capture one telemetry-to-alert trace.
- Run dependency, secret, and image scans; resolve critical release findings.
- Run one bounded k6 scenario and inspect the telemetry-range query plan.
- Document cold start, quota, sleep, session-loss, smoke, incident, and rollback behavior.

Exit gate

The no-card public HTTPS demo works after cold start; live incident and denial smoke pass; the deployed revision is identifiable; logs, four metrics, one trace, scans, measured performance, query-plan evidence, and rollback guidance are inspectable.

Free-tier limits that must remain visible

Service	Current constraint	Release treatment
Render Free	0.1 CPU, 512 MB, 750 instance-hours / workspace / month, idle sleep, ephemeral disk.	Bound JVM and pool, store no durable files locally, test cold start, and state suspension behavior.
Neon Free	0.5 GB storage / project, 100 CU-hours / project / month, scale-to-zero, bounded transfer.	Use bounded data and reset; avoid continuous simulator traffic; state quota behavior.
Application	Session and SSE subscribers are process-local.	Users sign in again after sleep / deploy; client refreshes authoritative durable state.

05 | V1.0 - RELEASE

v1.0 - Release

Remaining phase three of three: correct gaps, package evidence, and publish the immutable release.

Increment sequence

- Run the complete Definition of Done and create explicit defect / evidence-gap issues.
- Rehearse clean clone, deterministic seed / reset, and fresh-environment startup.
- Run complete local suites and public smoke against the release candidate.
- Correct defects only; add no capability that does not close a release gap.
- Finalize README, architecture, decisions, threat model, runbook, performance, and evidence index.
- Record the ninety-second demonstration, publish current screenshots, and tag immutable v1.0.

Exit gate

Every release criterion links to evidence; clean clone and the \$0 public demo work; v1.0 and the running image agree; limitations are current; and no unimplemented feature is claimed.

Release correction order

- Preserve tenant isolation and backend authorization.
- Preserve atomic / idempotent telemetry, durable retry, and explicit work conflict.
- Preserve critical automated journeys, reproducible startup, and truthful public status.
- Cut decorative polish, optional charts / filters, and secondary component breadth before core evidence.

06 | GIT, REVIEW, AND HOSTING CHANGE CONTROL

Git, review, and hosting change control

Repository history reflects real integration, while the current tree reflects the accepted plan.

Item	Convention
Issue	One coherent outcome or defect with IDs, checks, and security / tenant impact.
Branch	Short descriptive branch using the repository convention.
Commit	One coherent outcome after relevant checks pass.
Pull request	Outcome, contract IDs, design note, evidence, migration / deployment impact, and debt.
Main	Buildable; changes enter through reviewed pull requests and required checks.
Tag	Checkpoint only after its exit gate; v1.0 only after complete acceptance.
Release notes	Implemented behavior, direct evidence, known limitations, and next remaining phase.

Weekly review record

- What working behavior changed?
- What real test, integration, or deployment issue occurred?
- What architecture, schema, API, UI, or hosting decision changed after evidence?
- What direct test, CI, screenshot, measurement, or deployment evidence exists?
- What remains incomplete, and can the request, data, security, and failure paths be explained?

Hosting fallback rule

If the chosen free tier stops meeting the no-card requirement, do not start a subscription. Re-evaluate another no-card host, keep the provider-neutral container and smoke contracts, and mark public evidence pending until the replacement is proven.

Current source references

- Render free service limits: <https://render.com/docs/free>
- Render Blueprint fields and CI-gated deploy trigger: <https://render.com/docs/blueprint-spec>
- Render no-payment getting started guide: <https://render.com/docs/your-first-deploy>
- Neon Free plan quotas and no-card policy: <https://neon.com/pricing>

Publish a smaller truthful release only when the core journey and its security, reliability, zero-cost delivery, and evidence gates pass.